DATABASE MANAGEMENT SYSTEMS · ASSIGNMENT 9

Advanced SQL Queries Using Joins and Subqueries

Multi-table reports, summary statistics and related-record queries for FoodExpress

Project	FoodExpress – Online Food Delivery System	Submitted by	Dhruvan Karthik
DBMS	MySQL Community Server 8.0.46	GitHub	github.com/hidhruvan-prog
Database	`foodexpress`	Date	25 October 2026

Introduction

The reports in this assignment combine several tables of the `foodexpress` database to answer questions that the operations, marketing and finance teams of a food-delivery platform ask every day. They run on the data of Assignment 7, after the changes made by the DML statements in Assignment 8. Each report shows the SQL, the actual output from MySQL 8.0.46 and a short explanation. All statements are saved in `sql/06_advanced_queries.sql`.

Technique	Reports
INNER JOIN (3–6 tables)	R1, R2, R7 – R14, R17 – R19
LEFT JOIN / anti-join	R1, R3, R4, R5
Self join	R6
Joins with views	R1, R2, R7, R11
GROUP BY, HAVING, WITH ROLLUP, window functions (RANK, ROW_NUMBER)	R3, R5 – R10, R16, R17
Scalar subquery	R11, R13 (in SELECT and WHERE)
IN subquery (nested)	R12, R19
Correlated subquery	R13, R14, R15
EXISTS / NOT EXISTS	R14, R15
Derived table (subquery in FROM) and CTE (WITH)	R16, R17

1. Reports Using Joins

An INNER JOIN returns rows that have matches in both tables; a LEFT JOIN keeps every row of the left table and fills the missing right-hand columns with NULL. A self join joins a table to itself.

R1 Order register – the ten most recent orders (INNER and LEFT JOIN, 5 tables + view)

```sql
SELECT o.order_id,
       DATE_FORMAT(o.order_date, '%d-%b %H:%i') AS placed,
       c.full_name        AS customer,
       r.restaurant_name  AS restaurant,
       COALESCE(da.agent_name, '- not assigned -') AS rider,
       o.order_status,
       t.total_amount
FROM orders o
JOIN customers   c  ON c.customer_id   = o.customer_id
JOIN restaurants r  ON r.restaurant_id = o.restaurant_id
JOIN v_order_totals t ON t.order_id    = o.order_id
LEFT JOIN delivery_agents da ON da.agent_id = o.agent_id
ORDER BY o.order_date DESC
LIMIT 10;
```

mysql — foodexpress

```
mysql> SELECT o.order_id,
    ->        DATE_FORMAT(o.order_date, '%d-%b %H:%i') AS placed,
    ->        c.full_name        AS customer,
    ->        r.restaurant_name  AS restaurant,
    ->        COALESCE(da.agent_name, '- not assigned -') AS rider,
    ->        o.order_status,
    ->        t.total_amount
    -> FROM orders o
    -> JOIN customers   c  ON c.customer_id   = o.customer_id
    -> JOIN restaurants r  ON r.restaurant_id = o.restaurant_id
    -> JOIN v_order_totals t ON t.order_id    = o.order_id
    -> LEFT JOIN delivery_agents da ON da.agent_id = o.agent_id
    -> ORDER BY o.order_date DESC
    -> LIMIT 10;
+----------+--------------+----------------+--------------------+------------------+--------------+--------------+
| order_id | placed       | customer       | restaurant         | rider            | order_status | total_amount |
+----------+--------------+----------------+--------------------+------------------+--------------+--------------+
|       64 | 24-Oct 20:48 | Siddharth Jain | Punjabi Tadka      | - not assigned - | Placed       |       647.00 |
|       63 | 24-Oct 20:30 | Isha Desai     | Coastal Curry Co.  | - not assigned - | Preparing    |       706.00 |
|       62 | 24-Oct 20:10 | Rahul Verma    | Bombay Chaat Corner | Santosh Pawar    | Preparing    |        80.00 |
|       61 | 24-Oct 19:25 | Meera Pillai   | Napoli Slice       | Santosh Pawar    | Delivered    |      1645.00 |
|       60 | 24-Oct 17:00 | Priya Nair     | Dragon Wok         | Imran Khan       | Delivered    |       457.00 |
|       59 | 22-Oct 13:48 | Kabir Malhotra | Punjabi Tadka      | - not assigned - | Cancelled    |       309.00 |
|       58 | 20-Oct 21:54 | Siddharth Jain | Punjabi Tadka      | Ganesh More      | Delivered    |       608.00 |
|       57 | 20-Oct 13:17 | Rahul Verma    | Napoli Slice       | Deepak Yadav     | Delivered    |      2041.00 |
|       56 | 20-Oct 12:06 | Pooja Bhosale  | Punjabi Tadka      | Ganesh More      | Delivered    |       517.00 |
|       55 | 19-Oct 21:12 | Priya Nair     | Dragon Wok         | Imran Khan       | Delivered    |       517.00 |
+----------+--------------+----------------+--------------------+------------------+--------------+--------------+
10 rows in set (0.00 sec)
```

Explanation. Every order has a customer and a restaurant, so INNER JOINs are safe; the rider is joined with a LEFT JOIN because `agent_id` is NULL until a rider is assigned – an INNER JOIN would silently drop those orders. COALESCE replaces the NULL with readable text.

R2 Itemized invoice for order 38 (joins across 6 tables)

```sql
SELECT o.order_id, c.full_name AS customer,
       CONCAT(a.address_line, ', ', a.city, ' ', a.pincode) AS deliver_to,
       r.restaurant_name, p.payment_method, p.payment_status
FROM orders o
JOIN customers   c ON c.customer_id   = o.customer_id
JOIN addresses   a ON a.address_id    = o.address_id
JOIN restaurants r ON r.restaurant_id = o.restaurant_id
JOIN payments    p ON p.order_id      = o.order_id
WHERE o.order_id = 38;

SELECT m.item_name, oi.quantity, oi.price_at_order,
       oi.quantity * oi.price_at_order AS line_total
FROM order_items oi
JOIN menu_items m ON m.item_id = oi.item_id
WHERE oi.order_id = 38;

SELECT subtotal, delivery_fee, total_amount
FROM v_order_totals WHERE order_id = 38;
```

mysql — foodexpress

```
mysql> SELECT o.order_id, c.full_name AS customer,
    ->        CONCAT(a.address_line, ', ', a.city, ' ', a.pincode) AS deliver_to,
    ->        r.restaurant_name, p.payment_method, p.payment_status
    -> FROM orders o
    -> JOIN customers   c ON c.customer_id   = o.customer_id
    -> JOIN addresses   a ON a.address_id    = o.address_id
    -> JOIN restaurants r ON r.restaurant_id = o.restaurant_id
    -> JOIN payments    p ON p.order_id      = o.order_id
    -> WHERE o.order_id = 38;
+----------+-------------+-----------------------------------------------+--------------------------+----------------+----------------+
| order_id | customer    | deliver_to                                    | restaurant_name          | payment_method | payment_status |
+----------+-------------+-----------------------------------------------+--------------------------+----------------+----------------+
|       38 | Ananya Iyer | Manyata Tech Park, Nagavara, Bengaluru 560045 | Spice Route Biryani House | Card           | Completed      |
+----------+-------------+-----------------------------------------------+--------------------------+----------------+----------------+
1 row in set (0.00 sec)

mysql> SELECT m.item_name, oi.quantity, oi.price_at_order,
    ->        oi.quantity * oi.price_at_order AS line_total
    -> FROM order_items oi
    -> JOIN menu_items m ON m.item_id = oi.item_id
    -> WHERE oi.order_id = 38;
+--------------------+----------+----------------+------------+
| item_name          | quantity | price_at_order | line_total |
+--------------------+----------+----------------+------------+
| Chicken Dum Biryani |        1 |         289.00 |     289.00 |
| Mutton Dum Biryani  |        2 |         379.00 |     758.00 |
| Chicken 65          |        1 |         219.00 |     219.00 |
| Mirchi ka Salan     |        2 |          99.00 |     198.00 |
+--------------------+----------+----------------+------------+
4 rows in set (0.00 sec)

mysql> SELECT subtotal, delivery_fee, total_amount
    -> FROM v_order_totals WHERE order_id = 38;
+----------+--------------+--------------+
| subtotal | delivery_fee | total_amount |
+----------+--------------+--------------+
|  1464.00 |         0.00 |      1464.00 |
+----------+--------------+--------------+
1 row in set (0.00 sec)
```

Explanation. Three related queries build the bill a customer receives: a header (customer, address, restaurant, payment), the item lines with their line totals, and the totals from the view. Free delivery applies because the subtotal is at least ₹500.

R3 Restaurant activity, including restaurants with no orders (LEFT JOIN + GROUP BY)

```sql
SELECT r.restaurant_name, r.city,
       COUNT(o.order_id)                          AS orders,
       COALESCE(SUM(o.order_status = 'Delivered'), 0) AS delivered,
       COALESCE(SUM(o.order_status = 'Cancelled'), 0) AS cancelled,
       COALESCE(ROUND(100 * SUM(o.order_status = 'Cancelled')
                      / COUNT(o.order_id), 1), 0)     AS cancel_pct
FROM restaurants r
LEFT JOIN orders o ON o.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name, r.city
ORDER BY orders DESC;
```

mysql — foodexpress

```
mysql> SELECT r.restaurant_name, r.city,
    ->        COUNT(o.order_id)                          AS orders,
    ->        COALESCE(SUM(o.order_status = 'Delivered'), 0) AS delivered,
    ->        COALESCE(SUM(o.order_status = 'Cancelled'), 0) AS cancelled,
    ->        COALESCE(ROUND(100 * SUM(o.order_status = 'Cancelled')
    ->                       / COUNT(o.order_id), 1), 0)     AS cancel_pct
    -> FROM restaurants r
    -> LEFT JOIN orders o ON o.restaurant_id = r.restaurant_id
    -> GROUP BY r.restaurant_id, r.restaurant_name, r.city
    -> ORDER BY orders DESC;
+--------------------------+-----------+--------+-----------+-----------+------------+
| restaurant_name          | city      | orders | delivered | cancelled | cancel_pct |
+--------------------------+-----------+--------+-----------+-----------+------------+
| Punjabi Tadka            | Pune      |     13 |        10 |         2 |       15.4 |
| Spice Route Biryani House | Bengaluru |     11 |         9 |         2 |       18.2 |
| Dragon Wok               | Bengaluru |     10 |         9 |         1 |       10.0 |
| Napoli Slice             | Mumbai    |      7 |         6 |         1 |       14.3 |
| Bombay Chaat Corner      | Mumbai    |      7 |         6 |         0 |        0.0 |
| Coastal Curry Co.        | Mumbai    |      6 |         5 |         0 |        0.0 |
| Dosa Junction            | Bengaluru |      4 |         3 |         1 |       25.0 |
| Nizam's Kitchen          | Hyderabad |      3 |         3 |         0 |        0.0 |
| Green Bowl Salads        | Hyderabad |      3 |         2 |         1 |       33.3 |
| Sweet Tooth Desserts     | Pune      |      0 |         0 |         0 |        0.0 |
+--------------------------+-----------+--------+-----------+-----------+------------+
10 rows in set (0.00 sec)
```

Explanation. Because of the LEFT JOIN, Sweet Tooth Desserts appears with 0 orders. `COUNT(o.order_id)` is used instead of `COUNT(*)`, which would count the NULL-filled row as 1, and COALESCE turns the NULL sums of a restaurant without orders into 0.

R4 Registered customers who have never ordered (anti-join)

```sql
SELECT c.customer_id, c.full_name, c.email, c.registration_date
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

mysql — foodexpress

```
mysql> SELECT c.customer_id, c.full_name, c.email, c.registration_date
    -> FROM customers c
    -> LEFT JOIN orders o ON o.customer_id = c.customer_id
    -> WHERE o.order_id IS NULL;
+-------------+-------------+--------------------------+---------------------+
| customer_id | full_name   | email                    | registration_date   |
+-------------+-------------+--------------------------+---------------------+
|          14 | Divya Menon | divya.menon@example.com  | 2026-01-01 00:52:00 |
|          20 | Tanvi Kapoor | tanvi.kapoor55@example.com | 2026-10-13 10:00:00 |
+-------------+-------------+--------------------------+---------------------+
2 rows in set (0.00 sec)
```

Explanation. A LEFT JOIN followed by `IS NULL` on the right-hand key keeps only customers with no matching order – the audience for a "first order free delivery" campaign.

R5 Rider performance (LEFT JOIN, TIMESTAMPDIFF, aggregates)

```
SELECT da.agent_id, da.agent_name, da.city,
       COUNT(o.order_id)                                          AS deliveries,
       ROUND(AVG(TIMESTAMPDIFF(MINUTE, o.order_date, o.delivered_at)), 1)
                                                                  AS avg_minutes,
       MAX(TIMESTAMPDIFF(MINUTE, o.order_date, o.delivered_at))  AS slowest,
       ROUND(AVG(rv.agent_rating), 2)                             AS avg_rating
FROM delivery_agents da
LEFT JOIN orders o   ON o.agent_id  = da.agent_id AND o.order_status = 'Delivered'
LEFT JOIN reviews rv ON rv.order_id = o.order_id
GROUP BY da.agent_id, da.agent_name, da.city
ORDER BY deliveries DESC, avg_minutes;
```

mysql — foodexpress

```
mysql> SELECT da.agent_id, da.agent_name, da.city,
    ->        COUNT(o.order_id)                                          AS deliveries,
    ->        ROUND(AVG(TIMESTAMPDIFF(MINUTE, o.order_date, o.delivered_at)), 1)
    ->                                                                   AS avg_minutes,
    ->        MAX(TIMESTAMPDIFF(MINUTE, o.order_date, o.delivered_at))  AS slowest,
    ->        ROUND(AVG(rv.agent_rating), 2)                             AS avg_rating
    -> FROM delivery_agents da
    -> LEFT JOIN orders o   ON o.agent_id  = da.agent_id AND o.order_status = 'Delivered'
    -> LEFT JOIN reviews rv ON rv.order_id = o.order_id
    -> GROUP BY da.agent_id, da.agent_name, da.city
    -> ORDER BY deliveries DESC, avg_minutes;
+----------+---------------+-----------+------------+-------------+---------+------------+
| agent_id | agent_name    | city      | deliveries | avg_minutes | slowest | avg_rating |
+----------+---------------+-----------+------------+-------------+---------+------------+
|        5 | Deepak Yadav  | Mumbai    |         10 |        37.4 |      57 |       4.33 |
|        1 | Ravi Kumar    | Bengaluru |         10 |        40.3 |      55 |       4.50 |
|        9 | Ganesh More   | Pune      |         10 |        48.3 |      58 |       3.86 |
|        3 | Imran Khan    | Bengaluru |          8 |        44.6 |      58 |       4.00 |
|        4 | Santosh Pawar | Mumbai    |          4 |        45.8 |      58 |       5.00 |
|        7 | Srinivas Goud | Hyderabad |          3 |        44.3 |      58 |       3.50 |
|        2 | Manjunath S   | Bengaluru |          3 |        44.7 |      54 |       4.67 |
|        6 | Ajay Shinde   | Mumbai    |          3 |        45.7 |      58 |       3.33 |
|        8 | Mohammed Asif | Hyderabad |          2 |        48.5 |      56 |       NULL |
|       10 | Nikhil Jadhav | Pune      |          0 |        NULL |    NULL |       NULL |
+----------+---------------+-----------+------------+-------------+---------+------------+
10 rows in set (0.00 sec)
```

Explanation. The status filter is placed in the ON clause, not in WHERE, so riders with no deliveries (agent 10) are still listed with zeros and NULLs. Delivery time is computed from the two timestamps of each order.

R6 Dishes frequently ordered together (self join)

```
SELECT m1.item_name AS dish_1, m2.item_name AS dish_2,
       COUNT(*) AS times_together
FROM order_items a
JOIN order_items b ON b.order_id = a.order_id AND b.item_id > a.item_id
JOIN menu_items m1 ON m1.item_id = a.item_id
JOIN menu_items m2 ON m2.item_id = b.item_id
GROUP BY m1.item_name, m2.item_name
HAVING COUNT(*) >= 3
ORDER BY times_together DESC, dish_1;
```

mysql — foodexpress

```
mysql> SELECT m1.item_name AS dish_1, m2.item_name AS dish_2,
    ->        COUNT(*) AS times_together
    -> FROM order_items a
    -> JOIN order_items b ON b.order_id = a.order_id AND b.item_id > a.item_id
    -> JOIN menu_items m1 ON m1.item_id = a.item_id
    -> JOIN menu_items m2 ON m2.item_id = b.item_id
    -> GROUP BY m1.item_name, m2.item_name
    -> HAVING COUNT(*) >= 3
    -> ORDER BY times_together DESC, dish_1;
+----------------------+------------------+----------------+
| dish_1               | dish_2           | times_together |
+----------------------+------------------+----------------+
| Chicken 65           | Mirchi ka Salan  |              4 |
| Paneer Butter Masala | Sweet Lassi      |              4 |
| Butter Chicken       | Butter Naan      |              3 |
| Butter Naan          | Sweet Lassi      |              3 |
| Mutton Dum Biryani   | Chicken 65       |              3 |
| Mutton Dum Biryani   | Mirchi ka Salan  |              3 |
| Paneer Tikka         | Qubani ka Meetha |              3 |
| Pav Bhaji            | Masala Chaas     |              3 |
| Pepperoni Pizza      | Tiramisu         |              3 |
+----------------------+------------------+----------------+
9 rows in set (0.01 sec)
```

Explanation. `order_items` is joined to itself on the same order; the condition `b.item_id > a.item_id` lists each pair once. The result can drive “frequently bought together” suggestions and combo offers.

2. Summary Statistics Across Tables

Joins combined with GROUP BY, ROLLUP and window functions turn the raw tables into management reports.

R7 Restaurant leaderboard: revenue, average order value, rating and rank

```sql
SELECT r.restaurant_name,
       COUNT(*)                       AS delivered_orders,
       SUM(t.subtotal)                AS food_revenue,
       ROUND(AVG(t.total_amount), 2)  AS avg_order_value,
       vr.avg_rating, vr.review_count,
       RANK() OVER (ORDER BY SUM(t.subtotal) DESC) AS revenue_rank
FROM orders o
JOIN restaurants r          ON r.restaurant_id  = o.restaurant_id
JOIN v_order_totals t       ON t.order_id       = o.order_id
JOIN v_restaurant_ratings vr ON vr.restaurant_id = o.restaurant_id
WHERE o.order_status = 'Delivered'
GROUP BY r.restaurant_name, vr.avg_rating, vr.review_count
ORDER BY revenue_rank;
```

mysql — foodexpress

```
mysql> SELECT r.restaurant_name,
    ->        COUNT(*)                      AS delivered_orders,
    ->        SUM(t.subtotal)               AS food_revenue,
    ->        ROUND(AVG(t.total_amount), 2)  AS avg_order_value,
    ->        vr.avg_rating, vr.review_count,
    ->        RANK() OVER (ORDER BY SUM(t.subtotal) DESC) AS revenue_rank
    -> FROM orders o
    -> JOIN restaurants r          ON r.restaurant_id  = o.restaurant_id
    -> JOIN v_order_totals t       ON t.order_id       = o.order_id
    -> JOIN v_restaurant_ratings vr ON vr.restaurant_id = o.restaurant_id
    -> WHERE o.order_status = 'Delivered'
    -> GROUP BY r.restaurant_name, vr.avg_rating, vr.review_count
    -> ORDER BY revenue_rank;
+--------------------------+-----------------+--------------+-----------------+------------+--------------+--------------+
| restaurant_name          | delivered_orders | food_revenue | avg_order_value | avg_rating | review_count | revenue_rank |
+--------------------------+-----------------+--------------+-----------------+------------+--------------+--------------+
| Napoli Slice             |               6 |      7492.00 |         1253.67 |        5.0 |            3 |            1 |
| Spice Route Biryani House |               9 |      7215.00 |          812.78 |        4.4 |            8 |            2 |
| Punjabi Tadka            |              10 |      6937.00 |          702.60 |        4.1 |            8 |            3 |
| Dragon Wok               |               9 |      6519.00 |          740.67 |        3.0 |            6 |            4 |
| Nizam's Kitchen          |               3 |      2943.00 |          981.00 |        4.5 |            2 |            5 |
| Coastal Curry Co.        |               5 |      2560.00 |          536.00 |        5.0 |            3 |            6 |
| Green Bowl Salads        |               2 |      1992.00 |          996.00 |       NULL |            0 |            7 |
| Bombay Chaat Corner      |               6 |      1665.00 |          312.17 |        3.7 |            3 |            8 |
| Dosa Junction            |               3 |      1059.00 |          399.00 |        5.0 |            1 |            9 |
+--------------------------+-----------------+--------------+-----------------+------------+--------------+--------------+
9 rows in set (0.00 sec)
```

Explanation. Revenue comes from the `v_order_totals` view and ratings from `v_restaurant_ratings`, so no derived value is stored redundantly (Assignment 3). `RANK()` is a window function applied after grouping.

R8 City-wise business summary with grand total (GROUP BY … WITH ROLLUP)

```sql
SELECT COALESCE(r.city, 'ALL CITIES')        AS city,
       COUNT(*)                                  AS orders,
       SUM(o.order_status = 'Delivered')         AS delivered,
       SUM(o.order_status = 'Cancelled')         AS cancelled,
       SUM(CASE WHEN p.payment_status = 'Completed' THEN p.amount ELSE 0 END) AS collected,
       SUM(CASE WHEN p.payment_status = 'Refunded'  THEN p.amount ELSE 0 END) AS refunded
FROM orders o
JOIN restaurants r ON r.restaurant_id = o.restaurant_id
JOIN payments    p ON p.order_id      = o.order_id
GROUP BY r.city WITH ROLLUP;
```

mysql — foodexpress

```
mysql> SELECT COALESCE(r.city, 'ALL CITIES')        AS city,
    ->        COUNT(*)                                  AS orders,
    ->        SUM(o.order_status = 'Delivered')         AS delivered,
    ->        SUM(o.order_status = 'Cancelled')         AS cancelled,
    ->        SUM(CASE WHEN p.payment_status = 'Completed' THEN p.amount ELSE 0 END) AS collected,
    ->        SUM(CASE WHEN p.payment_status = 'Refunded'  THEN p.amount ELSE 0 END) AS refunded
    -> FROM orders o
    -> JOIN restaurants r ON r.restaurant_id = o.restaurant_id
    -> JOIN payments    p ON p.order_id      = o.order_id
    -> GROUP BY r.city WITH ROLLUP;
+------------+--------+-----------+-----------+-----------+----------+
| city       | orders | delivered | cancelled | collected | refunded |
+------------+--------+-----------+-----------+-----------+----------+
| Bengaluru  |     25 |        21 |         4 |  15178.00 |  1723.00 |
| Hyderabad  |      6 |         5 |         1 |   4935.00 |   557.00 |
| Mumbai     |     20 |        17 |         1 |  12155.00 |     0.00 |
| Pune       |     13 |        10 |         2 |   7026.00 |   628.00 |
| ALL CITIES |     64 |        53 |         8 |  39294.00 |  2908.00 |
+------------+--------+-----------+-----------+-----------+----------+
5 rows in set (0.00 sec)
```

Explanation. WITH ROLLUP adds a final super-aggregate row (city = NULL, shown as ALL CITIES) that totals every column.

R9 Top 5 best-selling dishes (delivered orders)

```sql
SELECT m.item_name, r.restaurant_name,
       SUM(oi.quantity)                    AS units_sold,
       SUM(oi.quantity * oi.price_at_order) AS revenue
FROM order_items oi
JOIN orders      o ON o.order_id      = oi.order_id
JOIN menu_items  m ON m.item_id       = oi.item_id
JOIN restaurants r ON r.restaurant_id = m.restaurant_id
WHERE o.order_status = 'Delivered'
GROUP BY m.item_id, m.item_name, r.restaurant_name
ORDER BY units_sold DESC, revenue DESC
LIMIT 5;
```

mysql — foodexpress

```
mysql> SELECT m.item_name, r.restaurant_name,
    ->        SUM(oi.quantity)                    AS units_sold,
    ->        SUM(oi.quantity * oi.price_at_order) AS revenue
    -> FROM order_items oi
    -> JOIN orders      o ON o.order_id      = oi.order_id
    -> JOIN menu_items  m ON m.item_id       = oi.item_id
    -> JOIN restaurants r ON r.restaurant_id = m.restaurant_id
    -> WHERE o.order_status = 'Delivered'
    -> GROUP BY m.item_id, m.item_name, r.restaurant_name
    -> ORDER BY units_sold DESC, revenue DESC
    -> LIMIT 5;
+-----------------+--------------------------+------------+---------+
| item_name       | restaurant_name          | units_sold | revenue |
+-----------------+--------------------------+------------+---------+
| Butter Naan     | Punjabi Tadka            |         13 |  715.00 |
| Chicken 65      | Spice Route Biryani House |         10 | 2190.00 |
| Tiramisu        | Napoli Slice             |          9 | 2241.00 |
| Mirchi ka Salan | Spice Route Biryani House |          9 |  891.00 |
| Garlic Bread    | Napoli Slice             |          8 | 1192.00 |
+-----------------+--------------------------+------------+---------+
5 rows in set (0.00 sec)
```

Explanation. Revenue uses `price_at_order` (the price actually charged), not the current menu price.

R10 Cuisine popularity in each city

```sql
SELECT r.city, r.cuisine_type,
       COUNT(DISTINCT o.order_id)   AS orders,
       COUNT(DISTINCT o.customer_id) AS customers,
       SUM(oi.quantity)             AS dishes
FROM orders o
JOIN restaurants r  ON r.restaurant_id = o.restaurant_id
JOIN order_items oi ON oi.order_id     = o.order_id
WHERE o.order_status <> 'Cancelled'
GROUP BY r.city, r.cuisine_type
ORDER BY r.city, orders DESC;
```

mysql — foodexpress

```
mysql> SELECT r.city, r.cuisine_type,
    ->        COUNT(DISTINCT o.order_id)   AS orders,
    ->        COUNT(DISTINCT o.customer_id) AS customers,
    ->        SUM(oi.quantity)             AS dishes
    -> FROM orders o
    -> JOIN restaurants r  ON r.restaurant_id = o.restaurant_id
    -> JOIN order_items oi ON oi.order_id     = o.order_id
    -> WHERE o.order_status <> 'Cancelled'
    -> GROUP BY r.city, r.cuisine_type
    -> ORDER BY r.city, orders DESC;
+-----------+--------------+--------+-----------+--------+
| city      | cuisine_type | orders | customers | dishes |
+-----------+--------------+--------+-----------+--------+
| Bengaluru | Chinese      |      9 |         6 |     31 |
| Bengaluru | Hyderabadi   |      9 |         5 |     35 |
| Bengaluru | South Indian |      3 |         3 |     11 |
| Hyderabad | Hyderabadi   |      3 |         3 |     17 |
| Hyderabad | Healthy      |      2 |         2 |      8 |
| Mumbai    | Street Food  |      7 |         3 |     23 |
| Mumbai    | Italian      |      6 |         2 |     28 |
| Mumbai    | Seafood      |      6 |         4 |     14 |
| Pune      | North Indian |     11 |         4 |     44 |
+-----------+--------------+--------+-----------+--------+
9 rows in set (0.00 sec)
```

Explanation. Because joining order_items repeats each order once per line, `COUNT(DISTINCT …)` is needed to count orders and customers correctly.

3. Reports Using Subqueries

A subquery is a SELECT inside another statement. It can return a single value (scalar), a list (used with IN), or a table (derived table in FROM). A correlated subquery refers to the outer row and is evaluated for each of them; EXISTS tests whether a subquery returns any row.

R11 Orders above the average order value (scalar subquery)

```sql
SELECT o.order_id, c.full_name, t.total_amount,
       ROUND(t.total_amount - (SELECT AVG(total_amount) FROM v_order_totals), 2)
                                              AS above_average_by
FROM orders o
JOIN customers c      ON c.customer_id = o.customer_id
JOIN v_order_totals t ON t.order_id    = o.order_id
WHERE t.total_amount > (SELECT AVG(total_amount) FROM v_order_totals)
ORDER BY t.total_amount DESC
LIMIT 10;
```

mysql — foodexpress

```
mysql> SELECT o.order_id, c.full_name, t.total_amount,
    ->        ROUND(t.total_amount - (SELECT AVG(total_amount) FROM v_order_totals), 2)
    ->                                                AS above_average_by
    -> FROM orders o
    -> JOIN customers c      ON c.customer_id = o.customer_id
    -> JOIN v_order_totals t ON t.order_id    = o.order_id
    -> WHERE t.total_amount > (SELECT AVG(total_amount) FROM v_order_totals)
    -> ORDER BY t.total_amount DESC
    -> LIMIT 10;
+----------+---------------+--------------+------------------+
| order_id | full_name     | total_amount | above_average_by |
+----------+---------------+--------------+------------------+
|       57 | Rahul Verma   |      2041.00 |          1317.47 |
|       24 | Aditya Joshi  |      1905.00 |          1181.47 |
|       61 | Meera Pillai  |      1645.00 |           921.47 |
|       44 | Pooja Bhosale |      1609.00 |           885.47 |
|       45 | Meera Pillai  |      1574.00 |           850.47 |
|       38 | Ananya Iyer   |      1464.00 |           740.47 |
|       20 | Meera Pillai  |      1336.00 |           612.47 |
|       50 | Aarav Sharma  |      1324.00 |           600.47 |
|       43 | Arjun Rao     |      1183.00 |           459.47 |
|       40 | Rohan Gupta   |      1155.00 |           431.47 |
+----------+---------------+--------------+------------------+
10 rows in set (0.00 sec)
```

Explanation. The inner query returns one number – the average order total – which the outer query compares each order against.

R12 Customers who have ordered Hyderabadi food (IN subquery)

```sql
SELECT customer_id, full_name, email
FROM customers
WHERE customer_id IN (
        SELECT o.customer_id
        FROM orders o
        WHERE o.order_status = 'Delivered'
          AND o.restaurant_id IN (SELECT restaurant_id
                                  FROM restaurants
                                  WHERE cuisine_type = 'Hyderabadi'))
ORDER BY full_name;
```

mysql — foodexpress

```
mysql> SELECT customer_id, full_name, email
    -> FROM customers
    -> WHERE customer_id IN (
    ->         SELECT o.customer_id
    ->         FROM orders o
    ->         WHERE o.order_status = 'Delivered'
    ->           AND o.restaurant_id IN (SELECT restaurant_id
    ->                                   FROM restaurants
    ->                                   WHERE cuisine_type = 'Hyderabadi'))
    -> ORDER BY full_name;
+-------------+--------------+--------------------------+
| customer_id | full_name    | email                    |
+-------------+--------------+--------------------------+
|           1 | Aarav Sharma | aarav.sharma@example.com |
|           2 | Ananya Iyer  | ananya.iyer@example.com  |
|          13 | Arjun Rao    | arjun.rao27@example.com  |
|          12 | Fatima Shaikh | fatima.shaikh@example.com |
|           5 | Karthik Reddy | karthik.reddy@example.com |
|           4 | Priya Nair   | priya.nair@example.com   |
|           3 | Rohan Gupta  | rohan.gupta@example.com  |
|          15 | Sai Teja     | sai.teja83@example.com   |
+-------------+--------------+--------------------------+
8 rows in set (0.00 sec)
```

Explanation. A nested subquery: the innermost query finds the Hyderabadi restaurants, the middle one finds customers with delivered orders from them, and the outer query lists those customers once each – useful for targeting a biryani festival promotion.

R13 Dishes priced above their own restaurant's average (correlated subquery)

```sql
SELECT r.restaurant_name, m.item_name, m.price,
       (SELECT ROUND(AVG(m2.price), 2)
        FROM menu_items m2
        WHERE m2.restaurant_id = m.restaurant_id) AS restaurant_avg
FROM menu_items m
JOIN restaurants r ON r.restaurant_id = m.restaurant_id
WHERE m.price > (SELECT AVG(m2.price)
                 FROM menu_items m2
                 WHERE m2.restaurant_id = m.restaurant_id)
ORDER BY r.restaurant_name, m.price DESC;
```

mysql — foodexpress

```
mysql> SELECT r.restaurant_name, m.item_name, m.price,
    ->        (SELECT ROUND(AVG(m2.price), 2)
    ->         FROM menu_items m2
    ->         WHERE m2.restaurant_id = m.restaurant_id) AS restaurant_avg
    -> FROM menu_items m
    -> JOIN restaurants r ON r.restaurant_id = m.restaurant_id
    -> WHERE m.price > (SELECT AVG(m2.price)
    ->                  FROM menu_items m2
    ->                  WHERE m2.restaurant_id = m.restaurant_id)
    -> ORDER BY r.restaurant_name, m.price DESC;
+--------------------------+--------------------------+--------+----------------+
| restaurant_name          | item_name                | price  | restaurant_avg |
+--------------------------+--------------------------+--------+----------------+
| Bombay Chaat Corner      | Pav Bhaji                | 140.00 |          73.00 |
| Bombay Chaat Corner      | Sev Puri                 |  80.00 |          73.00 |
| Coastal Curry Co.        | Surmai Fry               | 449.00 |         257.33 |
| Coastal Curry Co.        | Malvani Fish Curry       | 429.00 |         257.33 |
| Coastal Curry Co.        | Prawn Koliwada           | 399.00 |         257.33 |
| Dosa Junction            | Ghee Roast Dosa          | 129.00 |          89.00 |
| Dosa Junction            | Masala Dosa              | 109.00 |          89.00 |
| Dragon Wok               | Chicken Manchurian       | 249.00 |         189.00 |
| Dragon Wok               | Chilli Paneer            | 229.00 |         189.00 |
| Dragon Wok               | Chicken Fried Rice       | 219.00 |         189.00 |
| Green Bowl Salads        | Grilled Chicken Salad    | 329.00 |         259.00 |
| Green Bowl Salads        | Quinoa Power Bowl        | 299.00 |         259.00 |
| Napoli Slice             | Pepperoni Pizza          | 449.00 |         305.00 |
| Napoli Slice             | Margherita Pizza         | 349.00 |         305.00 |
| Napoli Slice             | Penne Arrabbiata         | 329.00 |         305.00 |
| Nizam's Kitchen          | Hyderabadi Chicken Biryani | 299.00 |         191.00 |
| Nizam's Kitchen          | Haleem                   | 249.00 |         191.00 |
| Nizam's Kitchen          | Paneer Tikka             | 239.00 |         191.00 |
| Punjabi Tadka            | Butter Chicken           | 329.00 |         191.67 |
| Punjabi Tadka            | Paneer Butter Masala     | 279.00 |         191.67 |
| Punjabi Tadka            | Dal Makhani              | 249.00 |         191.67 |
| Spice Route Biryani House | Mutton Dum Biryani       | 379.00 |         222.33 |
| Spice Route Biryani House | Chicken Dum Biryani      | 289.00 |         222.33 |
| Spice Route Biryani House | Veg Dum Biryani          | 229.00 |         222.33 |
| Sweet Tooth Desserts     | Belgian Chocolate Waffle | 199.00 |         139.00 |
| Sweet Tooth Desserts     | Red Velvet Pastry        | 149.00 |         139.00 |
+--------------------------+--------------------------+--------+----------------+
26 rows in set (0.00 sec)
```

Explanation. The subquery uses `m.restaurant_id` from the outer row, so it is re-evaluated per dish and compares each dish with the average of *its own* restaurant – the premium dishes of every menu.

R14 Delivered orders still waiting for a review (NOT EXISTS)

```sql
SELECT o.order_id, c.full_name, c.email, r.restaurant_name,
       DATE(o.delivered_at) AS delivered_on
FROM orders o
JOIN customers   c ON c.customer_id   = o.customer_id
JOIN restaurants r ON r.restaurant_id = o.restaurant_id
WHERE o.order_status = 'Delivered'
  AND NOT EXISTS (SELECT 1 FROM reviews rv WHERE rv.order_id = o.order_id)
ORDER BY o.delivered_at DESC;
```

mysql — foodexpress

```
mysql> SELECT o.order_id, c.full_name, c.email, r.restaurant_name,
    ->        DATE(o.delivered_at) AS delivered_on
    -> FROM orders o
    -> JOIN customers   c ON c.customer_id   = o.customer_id
    -> JOIN restaurants r ON r.restaurant_id = o.restaurant_id
    -> WHERE o.order_status = 'Delivered'
    ->   AND NOT EXISTS (SELECT 1 FROM reviews rv WHERE rv.order_id = o.order_id)
    -> ORDER BY o.delivered_at DESC;
+----------+---------------+-----------------------------+--------------------------+--------------+
| order_id | full_name     | email                       | restaurant_name          | delivered_on |
+----------+---------------+-----------------------------+--------------------------+--------------+
|       61 | Meera Pillai  | meera.pillai@example.com    | Napoli Slice             | 2026-10-24   |
|       60 | Priya Nair    | priya.nair@example.com      | Dragon Wok               | 2026-10-24   |
|       53 | Kabir Malhotra | kabir.malhotra@example.com | Punjabi Tadka            | 2026-10-17   |
|       48 | Rahul Verma   | rahul.verma@example.com     | Bombay Chaat Corner      | 2026-10-12   |
|       32 | Sai Teja      | sai.teja83@example.com      | Green Bowl Salads        | 2026-09-30   |
|       29 | Pooja Bhosale | pooja.bhosale@example.com   | Punjabi Tadka            | 2026-09-29   |
|       27 | Rahul Verma   | rahul.verma@example.com     | Napoli Slice             | 2026-09-27   |
|       25 | Fatima Shaikh | fatima.shaikh@example.com   | Nizam's Kitchen          | 2026-09-27   |
|       22 | Aditya Joshi  | aditya.joshi@example.com    | Bombay Chaat Corner      | 2026-09-25   |
|       20 | Meera Pillai  | meera.pillai@example.com    | Napoli Slice             | 2026-09-24   |
|       21 | Isha Desai    | isha.desai@example.com      | Coastal Curry Co.        | 2026-09-24   |
|       16 | Aarav Sharma  | aarav.sharma@example.com    | Dragon Wok               | 2026-09-21   |
|       12 | Fatima Shaikh | fatima.shaikh@example.com   | Green Bowl Salads        | 2026-09-17   |
|       11 | Sneha Kulkarni | sneha.kulkarni16@example.com | Dosa Junction           | 2026-09-17   |
|        8 | Rahul Verma   | rahul.verma@example.com     | Bombay Chaat Corner      | 2026-09-12   |
|        5 | Karthik Reddy | karthik.reddy@example.com   | Spice Route Biryani House | 2026-09-08   |
|        3 | Rohan Gupta   | rohan.gupta@example.com     | Dosa Junction            | 2026-09-03   |
|        2 | Rohan Gupta   | rohan.gupta@example.com     | Dragon Wok               | 2026-09-02   |
|        1 | Aditya Joshi  | aditya.joshi@example.com    | Coastal Curry Co.        | 2026-09-02   |
+----------+---------------+-----------------------------+--------------------------+--------------+
19 rows in set (0.00 sec)
```

Explanation. NOT EXISTS is true when the correlated subquery finds no review for the order. The list feeds the "rate your order" reminder e-mail.

R15 Restaurants with at least one poor review (EXISTS) – quality alert

```sql
SELECT r.restaurant_id, r.restaurant_name, r.city
FROM restaurants r
WHERE EXISTS (SELECT 1
              FROM orders o
              JOIN reviews rv ON rv.order_id = o.order_id
              WHERE o.restaurant_id = r.restaurant_id
                AND rv.food_rating <= 2);
```

mysql — foodexpress

```
mysql> SELECT r.restaurant_id, r.restaurant_name, r.city
    -> FROM restaurants r
    -> WHERE EXISTS (SELECT 1
    ->               FROM orders o
    ->               JOIN reviews rv ON rv.order_id = o.order_id
    ->               WHERE o.restaurant_id = r.restaurant_id
    ->                 AND rv.food_rating <= 2);
+---------------+-----------------+-----------+
| restaurant_id | restaurant_name | city      |
+---------------+-----------------+-----------+
|             3 | Dragon Wok      | Bengaluru |
+---------------+-----------------+-----------+
1 row in set (0.00 sec)
```

Explanation. EXISTS stops at the first matching review, so it is efficient for yes/no questions.

R16 Customer segments by lifetime spend (derived table + CASE)

```sql
SELECT segment,
       COUNT(*)                   AS customers,
       ROUND(AVG(orders), 1)      AS avg_orders,
       ROUND(AVG(spend), 2)       AS avg_spend,
       SUM(spend)                 AS total_spend
FROM (SELECT o.customer_id,
             COUNT(*)      AS orders,
             SUM(p.amount) AS spend,
             CASE WHEN SUM(p.amount) >= 3000 THEN 'Gold'
                  WHEN SUM(p.amount) >= 1500 THEN 'Silver'
                  ELSE 'Bronze' END AS segment
      FROM orders o
      JOIN payments p ON p.order_id = o.order_id
      WHERE p.payment_status = 'Completed'
      GROUP BY o.customer_id) AS customer_spend
GROUP BY segment
ORDER BY total_spend DESC;
```

mysql — foodexpress

```
mysql> SELECT segment,
    ->        COUNT(*)                   AS customers,
    ->        ROUND(AVG(orders), 1)      AS avg_orders,
    ->        ROUND(AVG(spend), 2)       AS avg_spend,
    ->        SUM(spend)                 AS total_spend
    -> FROM (SELECT o.customer_id,
    ->              COUNT(*)      AS orders,
    ->              SUM(p.amount) AS spend,
    ->              CASE WHEN SUM(p.amount) >= 3000 THEN 'Gold'
    ->                   WHEN SUM(p.amount) >= 1500 THEN 'Silver'
    ->                   ELSE 'Bronze' END AS segment
    ->       FROM orders o
    ->       JOIN payments p ON p.order_id = o.order_id
    ->       WHERE p.payment_status = 'Completed'
    ->       GROUP BY o.customer_id) AS customer_spend
    -> GROUP BY segment
    -> ORDER BY total_spend DESC;
+---------+-----------+------------+-----------+-------------+
| segment | customers | avg_orders | avg_spend | total_spend |
+---------+-----------+------------+-----------+-------------+
| Gold    |         5 |        4.8 |   4228.40 |    21142.00 |
| Silver  |         5 |        3.0 |   2042.40 |    10212.00 |
| Bronze  |         8 |        1.9 |    992.50 |     7940.00 |
+---------+-----------+------------+-----------+-------------+
3 rows in set (0.01 sec)
```

Explanation. The subquery in FROM (a derived table) first computes each customer's spend and segment; the outer query then summarizes the segments. The same result needs two levels of grouping, which a single query cannot do.

R17 Best-selling dish of every restaurant (CTE + ROW_NUMBER)

```
WITH dish_sales AS (
    SELECT m.restaurant_id, m.item_name,
           SUM(oi.quantity) AS units,
           ROW_NUMBER() OVER (PARTITION BY m.restaurant_id
                              ORDER BY SUM(oi.quantity) DESC, m.item_name) AS rn
    FROM order_items oi
    JOIN menu_items m ON m.item_id  = oi.item_id
    JOIN orders     o ON o.order_id = oi.order_id
    WHERE o.order_status <> 'Cancelled'
    GROUP BY m.restaurant_id, m.item_name
)
SELECT r.restaurant_name, ds.item_name AS best_seller, ds.units
FROM dish_sales ds
JOIN restaurants r ON r.restaurant_id = ds.restaurant_id
WHERE ds.rn = 1
ORDER BY ds.units DESC;
```

mysql — foodexpress

```
mysql> WITH dish_sales AS (
    ->     SELECT m.restaurant_id, m.item_name,
    ->            SUM(oi.quantity) AS units,
    ->            ROW_NUMBER() OVER (PARTITION BY m.restaurant_id
    ->                               ORDER BY SUM(oi.quantity) DESC, m.item_name) AS rn
    ->     FROM order_items oi
    ->     JOIN menu_items m ON m.item_id  = oi.item_id
    ->     JOIN orders     o ON o.order_id = oi.order_id
    ->     WHERE o.order_status <> 'Cancelled'
    ->     GROUP BY m.restaurant_id, m.item_name
    -> )
    -> SELECT r.restaurant_name, ds.item_name AS best_seller, ds.units
    -> FROM dish_sales ds
    -> JOIN restaurants r ON r.restaurant_id = ds.restaurant_id
    -> WHERE ds.rn = 1
    -> ORDER BY ds.units DESC;
+--------------------------+--------------------------+-------+
| restaurant_name          | best_seller              | units |
+--------------------------+--------------------------+-------+
| Punjabi Tadka            | Butter Naan              |    13 |
| Spice Route Biryani House | Chicken 65              |    10 |
| Napoli Slice             | Tiramisu                 |     9 |
| Dragon Wok               | Chicken Fried Rice       |     7 |
| Nizam's Kitchen          | Qubani ka Meetha         |     6 |
| Bombay Chaat Corner      | Masala Chaas             |     5 |
| Dosa Junction            | Masala Dosa              |     4 |
| Coastal Curry Co.        | Neer Dosa                |     4 |
| Green Bowl Salads        | Cold-Pressed Orange Juice |     3 |
+--------------------------+--------------------------+-------+
9 rows in set (0.00 sec)
```

Explanation. A common table expression (WITH) names the subquery. ROW_NUMBER numbers the dishes of each restaurant from best to worst seller, and the outer query keeps number 1. This is a "top-N per group" query.

R18 Price-change audit: order lines charged at an older price

```
SELECT oi.order_id, m.item_name,
       oi.price_at_order AS charged, m.price AS current_price,
       m.price - oi.price_at_order AS difference
FROM order_items oi
JOIN menu_items m ON m.item_id = oi.item_id
WHERE oi.price_at_order <> m.price
ORDER BY oi.order_id;
```

mysql — foodexpress

```
mysql> SELECT oi.order_id, m.item_name,
    ->        oi.price_at_order AS charged, m.price AS current_price,
    ->        m.price - oi.price_at_order AS difference
    -> FROM order_items oi
    -> JOIN menu_items m ON m.item_id = oi.item_id
    -> WHERE oi.price_at_order <> m.price
    -> ORDER BY oi.order_id;
+----------+-------------+---------+---------------+------------+
| order_id | item_name   | charged | current_price | difference |
+----------+-------------+---------+---------------+------------+
|        3 | Masala Dosa |   99.00 |        109.00 |      10.00 |
|       11 | Masala Dosa |   99.00 |        109.00 |      10.00 |
|       39 | Masala Dosa |   99.00 |        109.00 |      10.00 |
+----------+-------------+---------+---------------+------------+
3 rows in set (0.00 sec)
```

Explanation. After the Masala Dosa price change in Assignment 8 (Q23), earlier orders still record ₹99 while the menu shows ₹109. The report proves that historical invoices were not altered by the price update.

R19 Payment reconciliation: refunded and failed payments

```sql
SELECT p.payment_id, o.order_id, c.full_name, r.restaurant_name,
       p.payment_method, p.payment_status, p.amount
FROM payments p
JOIN orders      o ON o.order_id      = p.order_id
JOIN customers   c ON c.customer_id   = o.customer_id
JOIN restaurants r ON r.restaurant_id = o.restaurant_id
WHERE p.payment_status IN ('Refunded', 'Failed')
ORDER BY p.payment_status, p.amount DESC;

SELECT payment_status, COUNT(*) AS payments, SUM(amount) AS amount
FROM payments
WHERE order_id IN (SELECT order_id FROM orders WHERE order_status = 'Cancelled')
GROUP BY payment_status;
```

mysql — foodexpress

```
mysql> SELECT p.payment_id, o.order_id, c.full_name, r.restaurant_name,
    ->        p.payment_method, p.payment_status, p.amount
    -> FROM payments p
    -> JOIN orders      o ON o.order_id      = p.order_id
    -> JOIN customers   c ON c.customer_id   = o.customer_id
    -> JOIN restaurants r ON r.restaurant_id = o.restaurant_id
    -> WHERE p.payment_status IN ('Refunded', 'Failed')
    -> ORDER BY p.payment_status, p.amount DESC;
+------------+----------+---------------+--------------------------+----------------+----------------+---------+
| payment_id | order_id | full_name     | restaurant_name          | payment_method | payment_status | amount  |
+------------+----------+---------------+--------------------------+----------------+----------------+---------+
|         24 |       24 | Aditya Joshi  | Napoli Slice             | Cash           | Failed         | 1905.00 |
|         41 |       41 | Rohan Gupta   | Dragon Wok               | Cash           | Failed         |  846.00 |
|         36 |       36 | Ananya Iyer   | Spice Route Biryani House | Wallet         | Refunded       |  846.00 |
|         15 |       15 | Aarav Sharma  | Spice Route Biryani House | UPI            | Refunded       |  758.00 |
|         54 |       54 | Sai Teja      | Green Bowl Salads        | Card           | Refunded       |  557.00 |
|         52 |       52 | Siddharth Jain | Punjabi Tadka           | Wallet         | Refunded       |  319.00 |
|         59 |       59 | Kabir Malhotra | Punjabi Tadka           | Wallet         | Refunded       |  309.00 |
|         34 |       34 | Aarav Sharma  | Dosa Junction            | UPI            | Refunded       |  119.00 |
+------------+----------+---------------+--------------------------+----------------+----------------+---------+
8 rows in set (0.00 sec)

mysql> SELECT payment_status, COUNT(*) AS payments, SUM(amount) AS amount
    -> FROM payments
    -> WHERE order_id IN (SELECT order_id FROM orders WHERE order_status = 'Cancelled')
    -> GROUP BY payment_status;
+----------------+----------+---------+
| payment_status | payments | amount  |
+----------------+----------+---------+
| Refunded       |        6 | 2908.00 |
| Failed         |        2 | 2751.00 |
+----------------+----------+---------+
2 rows in set (0.00 sec)
```

Explanation. The first query lists every cancelled-order payment with the people involved; the second, using an IN subquery, confirms that all money taken for cancelled orders was refunded (cash orders were never collected, so their payment is Failed).

Conclusion

Joins reassemble the normalized tables into meaningful business views: order registers, invoices, rider and restaurant performance, city summaries and product-affinity pairs. LEFT JOINs make sure entities without activity are not lost, and views supply derived totals and ratings without storing redundant data. Subqueries – scalar, IN, correlated, EXISTS, derived tables and CTEs – answer questions that compare a row with a computed value or with related rows. Together they show that the normalized design of Assignment 3 does not limit reporting: every report needed by the platform can be produced from it with standard SQL.